

Reviewer Pack — Schur Coefficient Sum Ratio in Symmetric Powers of Permanent...

Entry d8795d1b709f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 20:40:50 UTC

Schur Coefficient Sum Ratio in Symmetric Powers of Permanent vs Determinant

Entry ID: d8795d1b709f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 20:40:50 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Circuit Size for Symmetric Polynomials

Statement:

For each $n \geq 2$, let S_n be the sum of Schur coefficients $c_\lambda(\text{perm}_n)$ for all partitions λ of n with $\lambda_1 \leq n^{\{1/2\}}$. Then, $S_n / S'_n > 2^{\{n/2\}}$, where S'_n is the analogous sum for $\det_n$.

Rationale (proposer's reasoning):

The permanent's symmetric structure under Schur-Weyl duality leads to a richer decomposition in symmetric powers, which could imply higher circuit complexity. The exponential separation suggests a fundamental asymmetry in their representation-theoretic properties.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a40e5afeae3eb0fd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations_with_replacement, permutations

def partitions(n):
    def partitions_recursive(n, max_partition=None):
        if n == 0:
            return [[]]
        result = []
        for i in range(1, min(n + 1, max_partition + 1)):
            for p in partitions_recursive(n - i, i):
                result.append([i] + p)
        return result
    return partitions_recursive(n)

def schur_coefficient(matrix, partition):
    n = len(matrix)
    if len(partition) != n:
        return 0

    def hook_length_formula(tableau):
        numerator = math.factorial(n)
        denominator = 1
        for i in range(n):
            for j in range(n):
                hook_length = (n - i) + (n - j) - partition[i] - partition[j] + 1
                denominator *= hook_length
        return numerator // denominator

    def is_standard_tableau(tableau):
        if len(set([tableau[i][j] for i in range(n) for j in range(n)])) != n * n:
            return False
        for i in range(n):
            if sorted(tableau[i]) != list(range(1, n + 1)):
                return False
            if sorted([tableau[j][i] for j in range(n)]) != list(range(1, n + 1)):
                return False
        return True
```

```

def generate_tableaux(partition):
    tableaux = []
    elements = list(range(1, n + 1))
    for perm in permutations(elements):
        tableau = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if perm[i * n + j] <= partition[i]:
                    tableau[i][j] = perm[i * n + j]
        if is_standard_tableau(tableau):
            tableaux.append(tableau)
    return tableaux

tableaux = generate_tableaux(partition)
return sum(hook_length_formula(t) for t in tableaux)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    S_n_total = 0
    S_prime_n_total = 0
    instances_tested = 0

    for n in n_values:
        perm_matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        det_matrix = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

        S_n = sum(schur_coefficient(perm_matrix, partition) for partition in partitions(n) if max(partition) <= n)
        S_prime_n = sum(schur_coefficient(det_matrix, partition) for partition in partitions(n) if max(partition) <= n)

        S_n_total += S_n
        S_prime_n_total += S_prime_n
        instances_tested += 2

    ratio = S_n_total / S_prime_n_total
    conjecture_holds = ratio > 2**((n_values[-1]-1)/2)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} <= 2^{(n_values[-1]-1)/2}"

    return {
        "metric_name": "Schur Coefficient Sum Ratio",
        "metric_value": ratio,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 307)) # Default to first 30 primes if no seeds are provided

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_102f931e.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_102f931e.py", line 80, in run_trial
    S_n = sum(schur_coefficient(perm_matrix, partition) for partition in partitions(n) if max(partition
                                             ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_102f931e.py", line 26, in partitions
    return partitions_recursive(n)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_102f931e.py", line 22, in partitions_re
    for i in range(1, min(n + 1, max_partition + 1)):
                          ~~~~~^
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in partitions_recursive function. The error suggests a missing initialization for max_partition variable. | next: Debug partitions_recursive function to handle max_partition initialization and fix the NoneType + int operation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	61291
2	propose	ollama_remote	qwen3:8b	0	0	94737
3	preregistration	ollama_remote	qwen3:8b	0	0	24442
4	novelty	ollama_remote	qwen3:8b	0	0	20539
5	novelty	ollama_remote	qwen3:8b	0	0	17023
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19233
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12005
8	judge	ollama_remote	qwen3:8b	0	0	17051

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 266322 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/d8795d1b709f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d8795d1b709f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d8795d1b709f.tar.gz` (if generated)